UNIT-I

Introduction to Scientific Programming and Python Basics

Scientific Programming

Scientific programming is , any time programming is used for science research. Using scientific programming, a researcher can increase the rate and reproducibility of their work exponentially. While humans are certainly better at some things than computers, doing massive calculations, storing data, and analyzing results are exactly what computers were designed to do. Scientists can use computers to automate processes that could otherwise be incredibly time-consuming, tedious, error-prone, and difficult for human researchers.

Python is widely used for scientific programming because it is easy to learn, has a simple syntax, and supports powerful libraries for numerical computing, data analysis, and visualization.

Overview of Scientific Computing

Scientific computing involves using computers to solve scientific problems through numerical analysis, simulations, and data processing. It is used in various fields like physics, biology, engineering, and finance. The key aspects of scientific computing include:

- **Numerical Computing**: Performing mathematical calculations on large datasets.
- **Data Analysis**: Processing and visualizing data to find patterns.
- **Simulation and Modeling**: Creating computer models to simulate real-world phenomena.
- **Optimization**: Finding the best solution among many possible choices.

Python is a preferred language for scientific computing because of libraries like **NumPy**, **SciPy**, **Matplotlib**, and **Pandas** that simplify these tasks.

Applications of Python

1. Scientific Computing & Mathematics

Python helps scientists and engineers solve complex problems quickly.

Uses:

- Solving equations
- Performing simulations
- Data analysis

Popular Libraries:

- **NumPy** (for numbers & matrices)
- **SciPy** (for scientific calculations)

2. Data Science & Analytics

Python is used to analyze and visualize large amounts of data.

Uses:

- Business forecasting
- Healthcare research
- Sports analytics

Popular Libraries:

- **Pandas** (for data handling)
- **Matplotlib** (for graphs)

3. Machine Learning & AI

Python is the leading language for AI.

Uses:

- Face recognition
- Fraud detection
- Self-driving cars

Popular Libraries:

- **TensorFlow / PyTorch** (for AI models)
- **Scikit-learn** (for machine learning)

General Programming

General programming is the process of writing code to create software that solves everyday problems, automates tasks, or builds applications for users across various domains like business, education, entertainment, and more.
It focuses on:
Building functional and interactive applications Writing logic to handle input/output, store data, and provide useful results.Designing user interfaces and ensuring a good user experience

Introduction to Python Programming

Python is a high-level, interpreted programming language that is beginner-friendly. It is widely used because:

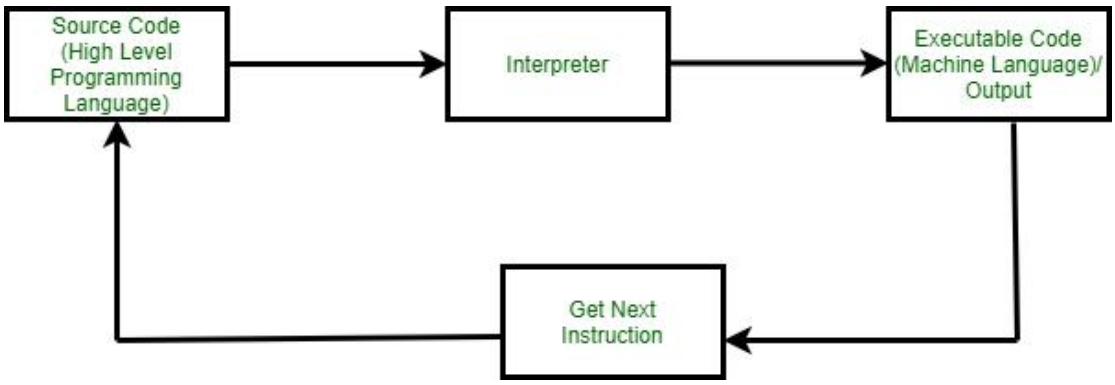
- It has simple and readable syntax.
- It supports multiple programming paradigms (procedural, object-oriented, functional).
- It has a large collection of libraries for scientific computing.

PYTHON BASICS

Python Interpreter

Python is an interpreted language developed by **Guido van Rossum** in the year of **1991**. Python is one of the most high-level languages used today because of its massive versatility and portable library & framework features. It is an interpreted language because it executes line-by-line instructions.
nterpreters are the computer program that will convert the source code or an high level language into intermediate code (machine level language).

It is also called translator in programming terminology. Interpreters executes each line of statements slowly. This process is called Interpretation. For example Python is an interpreted language, PHP, Ruby, and JavaScript.



Python Syntax

Syntax refers to the set of rules that defines how to write and organize code so that the Python interpreter can understand and run it correctly. These rules ensure that your code is structured, formatted, and error-free.

Indentation in Python

Python Indentation refers to the use of whitespace (spaces or tabs) at the beginning of code line. It is used to define the code blocks. Indentation is crucial in Python because, unlike many other programming languages that use braces "{}" to define blocks, Python uses indentation. It improves the readability of Python code, but on other hand it became difficult to rectify indentation errors. Even one extra or less space can leads to indentation error.

```
if 10 > 5:
    print("This is true!")
    print("I am tab indentation")

print("I have no indentation"
```

Comments in Python

Comments in Python are statements written within the code. They are meant to explain, clarify, or give context about specific parts of the code. The purpose of comments is to explain the working of a code, they have no impact on the execution or outcome of a program.

Python Single Line Comment

Single line comments are preceded by the "#" symbol. Everything after this symbol on the same line is considered a comment.

```
first_name = "Masrath"
last_name = "Parveen"  # assign last name

# print full name
print(first_name, last_name)
```

Python Multi-line Comment

Python doesn't have a specific syntax for multi-line comments. However, programmers often use multiple single-line comments, one after the other, or sometimes triple quotes (either ''' or """), even though they're technically string literals. Below is the example of multiline comment.

```
'''
Multi Line comment.
Code will print name.
'''

f_name = "Masrath Parveen"
print(f_name)
```

Python Variables

Variables in Python are essentially named references pointing to objects in memory. Unlike some other languages, you don't need to declare a variable's type explicitly in Python. Based on the value assigned, Python will dynamically determine the type.
In the below example, variable 'a' is initialize with integer value and variable 'b' with a string. Because of dynamic-types behavior, data type will be decide during runtime.

Rules for Naming Variables

To use variables effectively, we must follow Python's naming rules:

- Variable names can only contain letters, digits and underscores (_).
- A variable name cannot start with a digit.
- Variable names are case-sensitive (myVar and myvar are different).
- Avoid using Python keywords (e.g., if, else, for) as variable names.

Example

```
a = 56
print(type(a))
b = 'programming'
print(type(b))
```

Python Identifiers

In Python, identifiers are unique names that are assigned to variables, functions, classes, and other entities. They are used to uniquely identify the entity within the program. They should start with a letter (a-z, A-Z) or an underscore "_" and can be followed by letters, numbers, or underscores.

In the below example "first_name" is an identifier that store string value.

```
first_name = "Ram"
```

For naming of an identifier we have to follows some rules given below:

Identifiers can be composed of alphabets (either uppercase or lowercase), numbers (0-9), and the underscore character (_). They shouldn't include any special characters or spaces.

The starting character of an identifier must be an alphabet or an underscore.

Within a specific scope or namespace, each identifier should have a distinct name to avoid conflicts. However, different scopes can have identifiers with the same name without interference.

Python keywords

Keywords in Python are reserved words that have special meanings and serve specific purposes in the language syntax. They cannot be used as identifiers (names for variables, functions, classes, etc.). For instance, "for", "while", "if", and "else" are keywords and cannot be used as identifiers.

Below is the list of keywords in Python:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

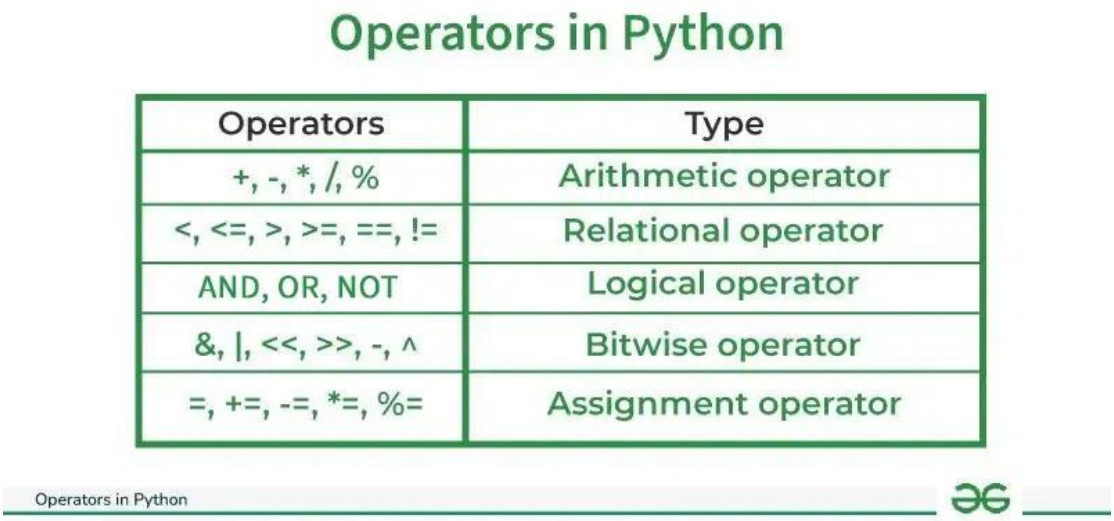
Operators in Python:

Operators in general are used to perform operations on values and variables. These are standard symbols used for logical and arithmetic operations.

- **OPERATORS:** These are the special symbols. Eg- + , * , /, etc.
- **OPERAND:** It is the value on which the operator is applied.

Types of Operators in Python

1. Arithmetic Operators
2. Comparison Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Identity Operators and Membership Operators



Arithmetic Operators in Python

Python Arithmetic operators are used to perform basic mathematical operations like addition, subtraction, multiplication and division.

```
# Variables
a = 11
b = 2
# Addition
print("Addition:", a + b)
# Subtraction
print("Subtraction:", a - b)
# Multiplication
print("Multiplication:", a * b)
# Division
print("Division:", a / b)
# Floor Division
print("Floor Division:", a // b)
# Modulus
print("Modulus:", a % b)
# Exponentiation
print("Exponentiation:", a ** b)
```

Comparison of Python Operators

In Python Comparison of Relational operators compares the values. It either returns True or False according to the condition.

```
a = 5
b = 10

print(a > b)
print(a < b)
print(a == b)
print(a != b)
print(a >= b)
print(a <= b)

OUTPUT
False
True
False
True
False
True
```

Logical Operators in Python

Python Logical operators perform Logical AND, Logical OR and Logical NOT operations. It is used to combine conditional statements.

The precedence of Logical Operators in Python is as follows:

1. Logical not
2. logical and
3. logical or

```
a = True
b = False
print(a and b)
print(a or b)
print(not a)

OUTPUT
False
True
False
```

Bitwise Operators in Python

Python Bitwise operators act on bits and perform bit-by-bit operations. These are used to operate on binary numbers.

Bitwise Operators in Python are as follows:

Bitwise AND

Bitwise OR

Bitwise NOT

Bitwise XOR

Bitwise Right Shift

Bitwise left Shift

```
a = 5
b = 3
print(a & b)
print(a | b)
print(~a)
print(a ^ b)
print(a >> 2)
print(a << 2)
```

```
OUTPUT FOR BITWISE OPERATORS
1
7
-6
6
1
20
```

Assignment Operators in Python

Python Assignment operators are used to assign values to the variables. This operator is used to assign the value of the right side of the expression to the left side operand.

```
a = 5
b = a
print(b)
b += a
print(b)
b -= a
print(b)
b *= a
print(b)
b //= a
print(b)
b %= a
print(b)
OUTPUT

5

10

5

25

5

0
```

Identity Operators in Python

In Python, is and is not are the identity operators both are used to check if two values are located on the same part of the memory. Two variables that are equal do not imply that they are identical.

is True if the operands are identical

is not True if the operands are not identical

```
a = 10
b = 10
print(a is b)
print(a is not b)
OUTPUT
True
False
```

Membership Operators in Python

In Python, in and not in are the membership operators that are used to test whether a value or variable is in a sequence.

in True if value is found in the sequence

not in True if value is not found in the sequence

```
x = 24
y = 20
list = [10, 20, 30, 40, 50]

if (x not in list):
    print("x is NOT present in given list")
else:
    print("x is present in given list")
if (y in list):
    print("y is present in given list")
else:
     print("y is NOT present in given list")

 OUTPUT
  x is NOT present in given list
  y is present in given list
```

Ternary Operator in Python

in Python, Ternary operators also known as conditional expressions are operators that evaluate something based on a condition being true or false.It simply allows testing a condition in a single line replacing the multiline if-else making the code compact.

Syntax : [on_true] if [expression] else [on_false]

```
a, b = 10, 20
min = a if a < b else b

print(min)
OUTPUT
10
```

Precedence and Associativity of Operators in Python

In Python, operators have different levels of precedence, which determine the order in which they are evaluated. When multiple operators are present in an expression, the ones with higher precedence are evaluated first. In the case of operators with the same precedence, their associativity comes into play, determining the order of evaluation

Precedence	Operators	Description	Associativity
1	**()**	Parentheses	Left to right
2	**x[index]**, **x[index:index]**	Subscription, slicing	Left to right
3	await x	Await expression	N/A
4	**	Exponentiation	Right to left
5	**+x**, **-x**, **~x**	Positive, negative, bitwise NOT	Right to left
6	*, @, /, //, **%**	Multiplication, matrix, division, floor division, remainder	Left to right

Precedence	Operators	Description	Associativity
7	+, -	Addition and subtraction	Left to right
8	<<, >>	Shifts	Left to right
9	&	Bitwise AND	Left to right
10	^	Bitwise XOR	Left to right
11	|	Bitwise OR	Left to right
12	**in**, **not in**, **is**, **is not**, <, <=, >, >=, !=, ==	Comparisons, membership tests, identity tests	Left to Right
13	**not x**	Boolean NOT	Right to left
14	and	Boolean AND	Left to right
15	or	Boolean OR	Left to right
16	if-else	Conditional expression	Right to left
17	lambda	Lambda expression	N/A
18	:=	Assignment expression (walrus operator)	Right to left

```
# Precedence of 'or' & 'and'
precedence of logical ‘and‘ is greater than the logical ‘or‘.
name = "Masrath"
age = 3

if name == "Masrath" or name == "Parveen" and age >= 5:
   print("Hello! Welcome.")
else:
    print("Good Bye!!")
    OUTPUT
    Hello! Welcome.

# Precedence of 'or' & 'and'
name = "Masrath"
age = 3

if (name == "Masrath" or name == "Parveen") and age >= 5:
   print("Hello! Welcome.")
else:
    print("Good Bye!!")
    OUTPUT
    Good Bye!!

#MORE EXAMPLES

# Left-right associativity
# 100 / 10 * 10 is calculated as
# (100 / 10) * 10 and not
# as 100 / (10 * 10)
print(100 / 10 * 10)

# Left-right associativity
# 5 - 2 + 3 is calculated as
# (5 - 2) + 3 and not
# as 5 - (2 + 3)
print(5 - 2 + 3)
# left-right associativity
print(5 - (2 + 3))
# right-left associativity
# 2 ** 3 ** 2 is calculated as
# 2 ** (3 ** 2) and not
# as (2 ** 3) ** 2
print(2 ** 3 ** 2)
OUTPUT
100.0
6
0
512
```

Type Casting in Python

Type Casting is the method to convert the Python variable datatype into a certain data type in order to perform the required operation by users. There can be two types of Type Casting in Python:

- Python Implicit Type Conversion
- Python Explicit Type Conversion

Implicit Type Conversion in Python

In this, method, Python converts the datatype into another datatype automatically. Users don't have to involve in this process.

```
# Python program to demonstrate
# implicit type Casting
# Python automatically converts
# a to int
a = 7
print(type(a))
# Python automatically converts
# b to float
b = 3.0
print(type(b))

# Python automatically converts
# c to float as it is a float addition
c = a + b
print(c)
print(type(c))

# Python automatically converts
# d to float as it is a float multiplication
d = a * b
print(d)
print(type(d))

OUTPUT:
<class 'int'>
<class 'float'>
10.0
<class 'float'>
21.0
<class 'float'>
```

Explicit Type Conversion in Python

This involves converting a data type explicitly using predefined functions like int(), float(), str(), etc., where the programmer specifically directs the type of conversion required.

Mainly type casting can be done with these data type functions:

Int(): Int() function take float or string as an argument and returns int type object.

float(): float() function take int or string as an argument and return float type object.

str(): str() function takes float or int as an argument and returns string type object.

Example:

```
# Python program to demonstrate
# Explicit type Casting
# int variable
a = 78
# typecast to float
n = float(a)
print(n)
print(type(n))
OUTPUT:
78.0
<class 'float'>
```

Input and Output in Python

Python provides two fundamental functions for output and input operations:

- print(): Used to display output to the console
- input(): Used to take input from the user.

Displaying output in Python

The print() function is used to display output on the screen.

Syntax:

print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)

Parameters:

- *objects: The values or expressions to be printed. Multiple objects can be printed, separated by commas.
- sep: The separator between multiple objects (default is a space ' ').
- end: The string that is printed at the end of output (default is a newline '\n').
- file: The output stream (default is sys.stdout, which means the console).
- flush: A boolean (True/False) to control buffer flushing (default is False).

different ways to use print() in Python:

Comma (,) Separator

- Prints multiple values separated by spaces.
- No need to convert data types.

String Concatenation (+)

- Joins strings using +.
- Requires str() conversion for non-string values.

% Formatting (Old Method)

- Uses placeholders (%s, %d, %f) for values.
- Outdated and less readable.

.format() Method

- Uses {} placeholders for values.
- More flexible than % formatting.

f-strings (Best & Recommended)

- Uses {} inside an f"" string.
- Fastest, most readable, and supports inline expressions.

```
name = "Masrath Parveen"
division = 3

# Using , (comma) separator
print("My name is", name, "and I am from division-",division,)

# Using + (String Concatenation)
print("My name is " + name + " and I am from division-" + str(division))

# Using % formatting (Old method)
print("My name is %s and I am from division-%d" % (name, division))

# Using .format() method
print("My name is {} and I am from division-{}".format(name, division))

# Using f-strings (Recommended)
print(f"My name is {name} and I am from division-{division}")
```

OUTPUT:

```
My name is Masrath Parveen and I am from division- 3
My name is Masrath Parveen and I am from division-3
My name is Masrath Parveen and I am from division-3
My name is Masrath Parveen and I am from division-3
My name is Masrath Parveen and I am from division-3
```

Taking input in Python

Python input() function is used to take user input. By default, it returns the user input in form of a string.

Syntax:

input(prompt)

Parameters:

- prompt: (Optional) A message displayed before the user enters input.

Converting Input Data Type

By default, input() returns data as a string. To convert it to a specific type, use type casting.

1. Integer Input
2. Float Input
3. Taking Multiple Inputs

```
# integer input
age = int(input("Enter your age: "))
print("Next year, you will be", age + 1)

# float input
price = float(input("Enter the price: "))
print("The price with tax is", price * 1.05)

# taking multiple input
a, b = input("Enter two numbers separated by space: ").split()
a, b = int(a), int(b)
print("Sum:", a + b)

OUTPUT:
Enter your age: 65
Next year, you will be 66
Enter the price: 43.9
The price with tax is 46.095
Enter two numbers separated by space: 2 3
Sum: 5
```

Escape Sequences In Python:

In Python, escape characters allow you to include special characters in strings that are otherwise difficult or impossible to type directly. They are preceded by a backslash (\) and help insert elements like newlines, tabs, quotes, or even a backslash itself. These characters are essential for formatting strings, handling file paths, and working with multiline text.

Escape Character	Description
\n	Newline – Moves the cursor to the next line.
\t	Tab – Adds a horizontal tab.
\\	Backslash – Inserts a literal backslash.

Escape Character	Description
\'	Single Quote – Inserts a single quote inside a single-quoted string.
\"	Double Quote – Inserts a double quote inside a double-quoted string.
\b	Backspace – Moves the cursor one position back, effectively deleting the last character.

```
print("Escape Sequences Demonstration:\n")  # \n for a new line
print("1. Newline:\nRoses are red,\nViolets are blue.")  # \n
print("2. Tab:\tPython makes coding fun!")  # \t
print("3. Backslash: To escape a backslash, use \\\\ like this.")  # \\
print('4. Single Quote: She said, \'Practice makes perfect.\')  # \'
print("5. Double Quote: The teacher said, \"Always check your work.\"")  # \"
print("6. Backspace: Mistak\bke corrected!")  # \b
```

OUTPUT:

```
Escape Sequences Demonstration:
1. Newline:
Roses are red,
Violets are blue.
2. Tab: Python makes coding fun!
3. Backslash: To escape a backslash, use \\ like this.
4. Single Quote: She said, 'Practice makes perfect.'
5. Double Quote: The teacher said, "Always check your work."
6. Backspace: Mistake corrected!
```

Python Primary Data Types

Python Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data.

The following are the standard or built-in data types in Python:

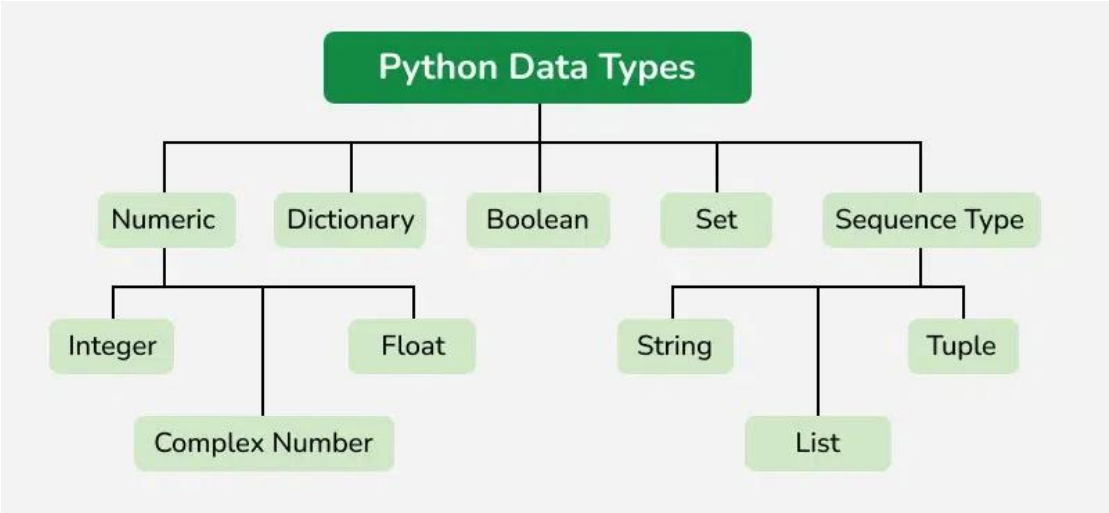
Numeric – int, float, complex

Sequence Type – string, list, tuple

Mapping Type – dict

Boolean – bool

Set Type – set, frozenset




```
# PYTHON BASIC DATA TYPES
# Integer
age = 25
print("Integer:", age)

# Float
price = 99.99
print("Float:", price)

# String
name = "Alice"
print("String:", name)

# Boolean
is_student = True
print("Boolean:", is_student)
```

```
# NoneType
empty_value = None
print("NoneType:", empty_value)

# Complex Number
complex_num = 3 + 4j
print("Complex:", complex_num)

#type of each variable

print(type(age),type(price),type(name),type(is_student),type(empty_value),type(comp
lex_num),sep="\n")
```

OUTPUT

```
Integer: 25
Float: 99.99
String: Alice
Boolean: True
NoneType: None
Complex: (3+4j)
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
<class 'complex'>
```

CONTROL STRUCTURES:

In Python, control structures dictate the order in which code is executed, enabling decision-making and repetition. The three primary types of control structures are: **sequential, selection, and repetition**. Sequential control structures execute statements in the order they appear. Selection structures, like if-else statements, allow code execution to branch based on conditions. Repetition structures, or loops (for and while), enable code blocks to be executed multiple times.

TYPES OF CONTROL STRUCTURES

1. Sequential
2. Selection
    a) If
    b) If-else
    c) If-elif-else
    d) Nested if-else
    e) Match cases.
3. Repetition.
    a) For loop
    b) While loop
    c) Nested loops(for and while)
4. Loop Control Statements:
    a) Break
    b) Continue
    c) Pass

1. Sequential Control Structures:

- These structures execute statements in the order they are written in the code.
- For example, if you have three lines of code, they will be executed one after the other, from top to bottom.
- This is the default control structure, where the program executes each statement sequentially without any branching or looping.

```
print("this gets executed first then x and then y is computed")
x = 5
y = x + 2
print(f"x={x}\tandy={y}")
print("this is the last statement that gets executed")

OUTPUT:
this gets executed first then x and then y is computed
x=5     andy=7
this is the last statement that gets executed
```

2. Selection Control Structures (Conditional Statements):

- These structures allow the program to make decisions based on specific conditions and execute different code blocks accordingly.
- The most common selection structure is the if-else statement.
- if statement: Checks if a condition is true, and if it is, it executes the code block associated with it.
- else statement: Executes the code block associated with it if the if condition is false.
- elif statement: (short for "else if") allows you to check multiple conditions and execute different code blocks based on the first true condition.
- Nested if-else:when we use if-else/if-elif-else statements inside another if or else block then they are called nested if -else. It is used when multiple conditions needed to be checked in a hierarchical manner.
- Match cases: The match statement in Python is a structural pattern matching statement that allows you to compare an expression against a series of patterns.

When a pattern matches the expression, the corresponding block of code is executed. It provides a more readable and powerful alternative to complex if-elif-else chains,

Syntax For Various Selection Control Structures

SYNTAX: IF

```
if condition:
    # code to execute if condition is true
```

SYNTAX: IF-ELSE

```
if condition:
    # code to execute if condition is true
else:
    # code to execute if condition is false
```

SYNTAX: IF-ELIF-ELSE

```
if condition1:
    # code to execute if condition1 is true
elif condition2:
    # code to execute if condition2 is true
else:
# code to execute if none of the conditions are true
```

SYNTAX: NESTED IF-ELSE

```
if condition1:
    # Code to execute if condition1 is True
    if condition2:
        # Code to execute if condition1 is True AND condition2 is True (nested if)
        statement(s)
    else:
        # Code to execute if condition1 is True AND condition2 is False (nested else)
        statement(s)
else:
    # Code to execute if condition1 is False
    if condition3:
        # Code to execute if condition1 is False AND condition3 is True (nested if)
        statement(s)
    else:
        # Code to execute if condition1 is False AND condition3 is False (nested else)
        statement(s)
```

SYNTAX: MATCH-CASES

```
match expression:
    case pattern_1:
        # Code to execute if expression matches pattern_1
        statement(s)
    case pattern_2:
        # Code to execute if expression matches pattern_2
        statement(s)
    case pattern_3 if condition:
        # Code to execute if expression matches pattern_3 AND condition is true
        statement(s)
    case _:  # Optional wildcard case (matches anything)
        # Code to execute if no preceding pattern matches
        statement(s)
```

Programs On Selection Control Structures

```
### 1. **Check if a Number is Positive, Negative, or Zero**
num = float(input("Enter a number: "))
if num > 0:
    print("The number is positive.")
elif num < 0:
    print("The number is negative.")
else:
    print("The number is zero.")

### 2. **Find the Largest of Three Numbers**
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a > b and a > c:
    print("The largest number is:", a)
elif b > c:
    print("The largest number is:", b)
else:
    print("The largest number is:", c)
```

```
### 3. **Check if a Year is a Leap Year**
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
    print(year, "is a leap year.")
else:
    print(year, "is not a leap year.")
### 4. **Check if a Number is Even or Odd**
num = int(input("Enter a number: "))
if num % 2 == 0:
    print("The number is even.")
else:
    print("The number is odd.")
### 5. **Grading System Based on Marks**
marks = int(input("Enter marks: "))
if marks >= 90:
    print("Grade: A")
elif marks >= 80:
    print("Grade: B")
elif marks >= 70:
    print("Grade: C")
elif marks >= 60:
    print("Grade: D")
else:
    print("Grade: F")
### 7. **Check if a Person is Eligible to Vote**
age = int(input("Enter your age: "))
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

```
### 8. **Find the Absolute Value of a Number**
num = float(input("Enter a number: "))
if num >= 0:
    print("Absolute value:", num)
else:
    print("Absolute value:", -num)
### 9. **Determine if a Triangle is Valid Based on Angles**
angle1 = int(input("Enter first angle: "))
angle2 = int(input("Enter second angle: "))
angle3 = int(input("Enter third angle: "))
if angle1 + angle2 + angle3 == 180:
    print("Valid Triangle")
else:
    print("Invalid Triangle")
### 10. **Check if a Number is Divisible by 5 and 11**
num = int(input("Enter a number: "))
if num % 5 == 0 and num % 11 == 0:
    print("The number is divisible by both 5 and 11.")
else:
    print("The number is not divisible by both 5 and 11.")
### **1. Check if a Number is a Multiple of 3 or 7**
num = int(input("Enter a number: "))
if num % 3 == 0 and num % 7 == 0:
    print("The number is a multiple of both 3 and 7.")
elif num % 3 == 0:
    print("The number is a multiple of 3.")
elif num % 7 == 0:
    print("The number is a multiple of 7.")
else:
    print("The number is not a multiple of 3 or 7.")
```

```
### **2. Check if a Triangle is Equilateral, Isosceles, or Scalene**
a = int(input("Enter first side: "))
b = int(input("Enter second side: "))
c = int(input("Enter third side: "))
if a == b == c:
    print("Equilateral Triangle")
elif a == b or b == c or a == c:
    print("Isosceles Triangle")
else:
    print("Scalene Triangle")
### **3. Find the Smallest of Three Numbers**
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))
if a < b and a < c:
    print("The smallest number is:", a)
elif b < c:
    print("The smallest number is:", b)
else:
    print("The smallest number is:", c)
### **4. Check if a Character is Uppercase or Lowercase**
char = input("Enter a character: ")
if 'A' <= char <= 'Z':
    print("Uppercase Letter")
elif 'a' <= char <= 'z':
    print("Lowercase Letter")
else:
    print("Not an Alphabet")
### **5. Check if a Number is Positive, Negative, or Zero (Without `elif`)**
num = int(input("Enter a number: "))
if num > 0:
    print("Positive Number")
```

```
if num < 0:
    print("Negative Number")
if num == 0:
    print("Zero")
### **6. Categorize Age Groups**
age = int(input("Enter your age: "))
if age <= 12:
    print("Child")
elif age <= 19:
    print("Teenager")
elif age <= 59:
    print("Adult")
else:
    print("Senior Citizen")
### **7. Check if a Number is a Two-Digit Number**
num = int(input("Enter a number: "))
if 10 <= num <= 99 or -99 <= num <= -10:
    print("Two-digit number")
else:
    print("Not a two-digit number")
```

ON NESTED IF-ELSE:

```
# Checking student grade based on marks (NESTED IF-ELSE)

marks=int(input("enter the marks scored: "))
if marks> 100:
    print("enter valid marks!")
elif (marks>=50) and (marks<=100):
    if marks>=90:
        print("Grade:A")
    elif marks>=80:
        print("Grade:B")
    elif marks >=70:
```

```
        print("Grade:C")
    else:
        print("Grade:D")

else:
    print("Fail")

# ------------------------------------------------
# Checking whether a number is even or odd, and also checking divisibility by
          5(NESTED)

num=int(input("enter a positive number:"))
if num%2==0:
    if num%5==0:
        print("the number is an even number and also divisible by 5")
    else:
        print("the number is an even number but its not divisible by 5")
else:
    print("its an odd number!")

# ------------------------------------------------------------
# Checking whether a Triangle is  equilateral or isosceles or scalene(NESTED)
a, b, c = map(float, input("Enter three sides: ").split())
A, B, C = map(float, input("Enter three angles: ").split())
if A+B+C ==180:
    if a+b>c and b+c>a and a+c>b :
        if a==b==c :
            print("equilateral triangle")
        elif a==b or b==c or c==a :
            print("isosceles")
        else:
            print("scalene")
    else:
```

```
        print("the sum of any two sides must be greater than the third side")
else:
    print("the angles must be 180 degree")
```

ON MATCH-CASES(use python versions >=3.10):

```
# Simple example using match-case

value=6.8
match 2+4:
    case "3":
        print("you entered a str 3!")
    case 6:
        print("its 6!")
    case _:
        print("invalid")

# print day using match-case

day = "Friday"

match day:
    case "Monday":
        print("Start of the work week.")
    case "Friday":
        print("Almost the weekend!")
    case "Sunday":
        print("Weekend vibes.")
    case _:
        print("Just another day.")
```

```
# Simple calculator using match-case

num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))
operator = input("Enter operator (+, -, *, /): ")
match operator:
    case "+":
        print("Result:", num1 + num2)
    case "-":
        print("Result:", num1 - num2)
    case "*":
        print("Result:", num1 * num2)
    case "/":
        if num2 != 0:
            print("Result:", num1 / num2)
        else:
            print("Error: Cannot divide by zero")
    case _:
        print("Invalid operator")
```

When to use which conditional statement:

- If there are simple conditions then go for (basic if-else/if-elif-else)
- If there are multiple conditions and if the condition depends on the previous condition go for (nested). it is more prone to errors because of indentation.
- For matching patterns,menu driven programs,multiple fixed cases go for (match cases), as it is more organized, provides more elegant & readable then nested.

3. Repetition Control Structures (Loops):

These structures allow the program to execute a block of code repeatedly until a certain condition is met. Python has two primary types of loops: for loops and while loops.

ITERATION: Repetition of a process. Repeating a block of code until a specific condition is met for certain number of times

While Loop in Python

- In Python, a while loop is used to execute a block of statements repeatedly until a given condition is satisfied. When the condition becomes false, the line immediately after the loop in the program is executed.
- All the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

For Loop in Python

A for loop in Python is used to iterate over a sequence (such as a list, tuple, string, or range) or other iterable objects. It executes a block of code for each item in the sequence.

- The for loop iterates through each item in the given sequence.
- For each item, the variable is assigned the value of the current item.
- The code block inside the loop is then executed using this value of the variable.
- This process continues until all items in the sequence have been processed.

while-else Loop

Python allows an optional else block to be used with a while loop. The else block is executed when the while loop finishes normally (i.e., the condition becomes false). If the loop is terminated prematurely by a break statement, the else block is not executed.

for-else Loop

Similar to while loops, for loops can also have an optional else block. The else block in a for loop is executed after the loop has completed all its iterations. However, if the for loop is terminated prematurely by a break statement, the else block is not executed.

Nested loops:

Python allows you to put one loop inside another loop. This is known as nested loops. Nested loops are useful when you need to iterate over items within items, such as elements in a list of lists or characters in a string for each item in a list.

SYNTAX OF EACH:

```
SYNTAX :while loop
while condition:
    # Code to be executed as long as the condition is true
SYNTAX :for loop
for variable in sequence:
    # Code to be executed for each item in the sequence
SYNTAX :while-else loop
while condition:
    # Code to be executed while the condition is true
else:
    # Code to be executed when the loop finishes normally
SYNTAX :for-else loop
for variable in sequence:
    # Code to be executed for each item in the sequence
else:
    # Code to be executed when the loop finishes normally
```

```
SYNTAX : :NESTED FOR LOOP
for iterator_var in sequence:
   for iterator_var in sequence:
      statements(s)
   statements(s)

SYNTAX :NESTED WHILE LOOP
while expression:
   while expression:
      statement(s)
   statement(s)
```

EXAMPLE PROGRAMS ON ALL TYPES OF LOOPS:

```
# WHILE LOOP
a = 0
while (a < 3):
    a = a+ 1
    print("hi this is masrath!")
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
hi this is masrath!
hi this is masrath!
hi this is masrath!
#FOR LOOP
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
     print(fruit)
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
apple
banana
Cherry
```

WHILE ELSE LOOP

```
cnt = 0
while (cnt < 3):
    cnt = cnt + 1
    print("Hello DIV-3&6!")
else:
     print("In Else Block")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Hello DIV-3&6!
Hello DIV-3&6!
Hello DIV-3&6!
In Else Block
```

FOR-ELSE LOOP

```
list = ["DIV-3", "AND", "DIV-6"]
for index in range(len(list)):
    print(list[index])
else:
    print("Inside Else Block")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
DIV-3
AND
DIV-6
Inside Else Block
```

```
### **1. Print Numbers from 1 to 10 (Using `for` Loop)**

for i in range(1, 11):
    print(i)

### **2. Print Even Numbers from 1 to 20 (Using `while` Loop)**

i = 2
while i <= 20:
    print(i)
    i += 2

### **3. Sum of First N Natural Numbers (User Input)**

n = int(input("Enter a number: "))
sum = 0

for i in range(1, n + 1):
    sum += i

print("Sum of first", n, "natural numbers is:", sum)

### **4. Multiplication Table of a Given Number**

num = int(input("Enter a number: "))

for i in range(1, 11):
    print(num, "x", i, "=", num * i)

### **5. Reverse a Number (Using `while` Loop)**

num = int(input("Enter a number: "))
```

```
rev = 0

while num > 0:
    digit = num % 10
    rev = rev * 10 + digit
    num //= 10

print("Reversed number:", rev)

### **6. Factorial of a Number**

n = int(input("Enter a number: "))
fact = 1

for i in range(1, n + 1):
    fact *= i

print("Factorial of", n, "is:", fact)

### **7. Check if a Number is Prime**
num = int(input("Enter a number: "))
is_prime = True

if num > 1:
    for i in range(2, num):
        if num % i == 0:
            is_prime = False
            break

if is_prime and num > 1:
    print(num, "is a prime number")
else:
    print(num, "is not a prime number")
```

```
### **8. Fibonacci Series (First N Terms)**

n = int(input("Enter number of terms: "))
a, b = 0, 1

for _ in range(n):
    print(a, end=" ")
    a, b = b, a + b

### **9. Print a Right-Angled Triangle Pattern**

rows = int(input("Enter number of rows: "))

for i in range(1, rows + 1):
    print("*" * i)

### **10. Count Digits in a Number**

num = int(input("Enter a number: "))
count = 0

while num > 0:
    num //= 10
    count += 1

print("Number of digits:", count)
```

4. Loop Control Statements:

- These statements can be used within loops to control their execution.
- **break**: Terminates the loop prematurely, exiting it.
- **continue**: Skips the current iteration of the loop and proceeds to the next one.
- **pass:** Acts as a placeholder, meaning it does nothing but is necessary to satisfy the syntax when a code block is required but no action needs to be taken.

```
BREAK:

for i in "Masrath":
    if i=="t":
        break
     print(i)


OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
M
a
s
r
A

CONTINUE

for i in "Masrath":
    if i=="t":
        continue
    print(i)

OUTPUT:

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
M
a
s
r
a
H
```

```
PASS

for i in range(5):
    if i==3:
        pass
    else:
        print(i)

OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
0
1
2
4
```

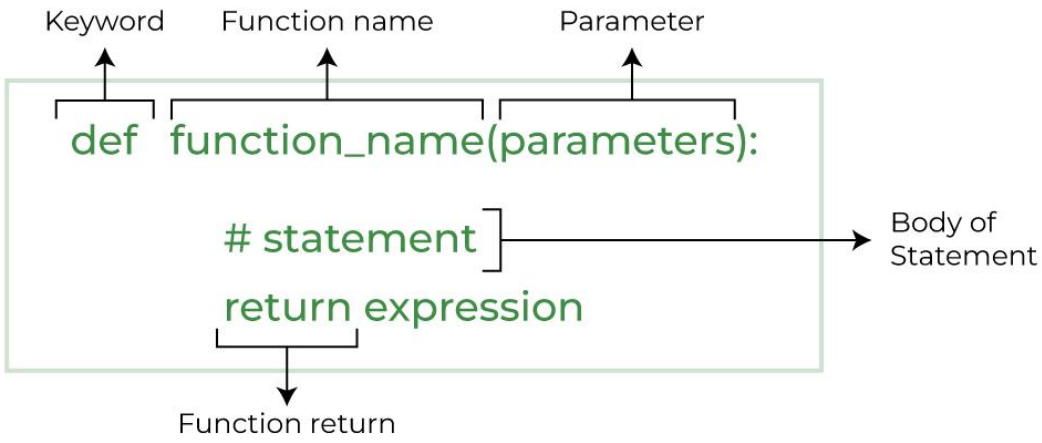
WHEN TO USE WHICH LOOP:

- Choose for loops when you know the sequence you need to iterate over or when you know the number of iterations in advance (or can determine) the number of iterations.
- Choose while loops when the iteration depends on a condition that might change within the loop, and you don't know the exact number of iterations beforehand.
- Use the else block with for or while when you need to execute code specifically if the loop completes normally without a break.
- Use nested loops when you need to process elements in data structures that have multiple dimensions, like lists of lists (representing matrices), or when you need to consider all possible pairs or combinations of items from multiple sequences.

FUNCTIONS IN PYTHON

In Python, a function is a reusable block of code.A function is a sequence of instructions that performs a specific task. A function is a block of code that can run when it is called. A function can have input arguments, which are made available to it by the user, the entity calling the function. Functions also have output parameters, which are the results of the function that the user expects to receive once the function has completed its task.

Functions help organize code, making it more readable and easier to maintain. They can take inputs (arguments), process them, and return outputs.



Why/When To Use Functions?

To avoid repetition of code.

To organize code logically.

To reuse code with different inputs.

To simplify debugging and testing.

Types of Functions in Python

Built-in library function: These are Standard functions in Python that are available to use.

User-defined function: We can create our own functions based on our requirements.

BUILT-IN FUNCTIONS:

len()

Description: Returns the number of items in an object (like a list, string, tuple, etc.)

Syntax: len(object)

```
name = "MASRATH PARVEEN" #Including white space
print("Length of the string is:", len(name))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Length of the string is: 15
```

type()

Description: Returns the data type of an object.

Syntax: type(object)

```
number = 10
print("The type of number is:", type(number))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
The type of number is: <class 'int'>
```

input()

Description: Takes input from the user as a string.

Syntax: input(prompt)

```
name = input("Enter your name: ")
print("Hello", name)
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Enter your name: Masrath Parveen
Hello Masrath Parveen
```

sum()

Description: Returns the sum of all items in an iterable (e.g., list or tuple).

Syntax: sum(iterable)

```
marks = [85, 90, 78]
total = sum(marks)
print("Total marks:", total)
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Total marks: 253
```

max() and min()

Description: Returns the largest (max) or smallest (min) item from an iterable.

Syntax: max(iterable) or min(iterable)

```
numbers = [12, 45, 67, 23]
print("Maximum number:", max(numbers))
print("Minimum number:", min(numbers))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Maximum number: 67
Minimum number: 12
```

Map()

map() is a built-in function that allows you to apply a specific function to every item in an iterable (like a list, tuple, string, etc.)

map(function, iterables)

```

x = map(len, ('apple', 'banana', 'cherry'))
print(list(x))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
[5, 6, 6]
```

Parameter Values

Parameter	Description
function	Required. The function to execute for each item
iterable	Required. A sequence, collection or an iterator object. You can send as many iterables as you like, just make sure the function has one parameter for each iterable.

Range()

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Syntax

range(start, stop, step)

Parameter Values

Parameter	Description
start	Optional. An integer number specifying at which position to start. Default is 0
stop	Required. An integer number specifying at which position to stop (not included).
step	Optional. An integer number specifying the incrementation. Default is 1

range (stop)

```
for i in range(5):
    print(i, end=" ")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
0 1 2 3 4
```

range (start, stop)

```
for i in range(5,20):
   print(i, end=" ")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
5 6 7 8 9 10 11 12 13 14 15 16 17 18 19
```

range (start, stop, step)

```
for i in range(0,10,2):
   print(i, end=" ")
```

OUTPUT:

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
0 2 4 6 8
```

USER -DEFINED FUNCTIONS:

We can define our own functions. A function can be specified in several ways. Here we will introduce the most common way to define a function which can be specified using the keyword def, as showing in the following:

```
def function_name(argument_1, argument_2, ...):
    '''
    Descriptive String
    '''
    # comments about the statements
    function_statements

    return output_parameters (optional)
```

We could see that defining a Python function need the following two components:
Function header: A function header starts with a keyword def, followed by a pair of parentheses with the input arguments inside, and ends with a colon (:)
Function Body: An indented (usually four white spaces) block to indicate the main body of the function. It consists 3 parts:
Descriptive string: A string that describes the function that could be accessed by the help() function or the question mark. You can write any strings inside, it could be multiple lines.
Function statements: These are the step by step instructions the function will execute when we call the function. You may also notice that there is a line starts with '#', this is a comment line, which means that the function will not execute it.
Return statements: A function could return some parameters after the function is called, but this is optional, we could skip it. Any data type could be returned, even a function, we will explain more later.

PROBLEM: Define a function named my_adder to take in 3 numbers and sum them.

```
def my_adder(a, b, c):
    """
    function to sum the 3 numbers
    Input: 3 numbers a, b, c
    Output: the sum of a, b, and c
    author:Masrath parveen
    date:23-04-2025
    """

    # this is the summation
    out = a + b + c
    return out
#calling the function my_adder and store the returned value in d
d = my_adder(1, 2, 3)
# printing the value stored in d
print(d)
# calling the same function but with different values
d = my_adder(4, 5, 6)
print(d)
help(my_adder)
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
6
15
Help on function my_adder in module __main__:
my_adder(a, b, c)
    function to sum the 3 numbers
    Input: 3 numbers a, b, c
    Output: the sum of a, b, and c
    author:Masrath parveen
     date:23-04-2025
```

```
d = my_adder(5 + 2, 3 * 4, 12 / 6)  # mathematical expressions as the input
to my_adder
print(d) output 21.0
```

Default Arguments

A default argument is a parameter that assumes a default value if a value is not provided in the function call for that argument.

```
def person(name,age=17):
    print(f"Name:{name},Age:{age}")
person("John")
person("Mike",25)
person("harry")
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Name:John,Age:17
Name:Mike,Age:25
Name:harry,Age:17
```

Arbitrary Keyword Arguments

In Python Arbitrary Keyword Arguments, *args, and **kwargs can pass a variable number of arguments to a function using special symbols. There are two special symbols:

***args in Python (Non-Keyword Arguments):**

Used to pass a variable number of non-keyword arguments.

Internally, Python stores them as a tuple.

Non-keyword arguments (also called positional arguments):

You just pass the value.

Example 1: Non-keywords argument

```
def add(*numbers):
    return sum(numbers)
print(add(1,2,3,4,5))
```

```
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
15

def multiply(factor=2,*args):
    for num in args:
        print(f"{factor}*{num}={factor*num}")
multiply(3,5,10,15)
multiply(4,5)
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
3*5=15
3*10=30
3*15=45
4*5=20
```

****kwargs in Python (Keyword Arguments):**

Used to pass a variable number of keyword arguments.

Internally stored as a dictionary.

Keyword arguments:

You pass the name of the parameter with a value.

Order doesn't matter, because Python uses the parameter names.

Example 2: keywords argument

```
def my_func(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} = {value}")
my_func(name="Alice", age=25, city="Paris")
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
name = Alice
age = 25
city = Paris
```

Lambda function/Anonymous Functions in Python

Python Lambda Functions are anonymous functions means that the function is without a name. As we already know the def keyword is used to define a normal function in Python. Similarly, the lambda keyword is used to define an anonymous function in Python.

Syntax: lambda arguments : expression

```
# Using lambda
square = lambda x: x ** 2
print(square(3))

# Using def
def square(x):
    return x ** 2
print(square(3))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
9
9

add=lambda a,b:a+b
print(add(2,3))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
5
```

When to use lambda function:

- Lambda functions are small one time functions.
- When you want to avoid defining a full function.
- When you want to perform simple operations
- Used for quick and simple tasks without naming the function.

scope and lifetime of variables in python

1. scope — Where the variable is accessible
2. Lifetime — How long the variable exists

A variable's lifetime is the period during which the variable exists in memory.

Local variables are created when a function starts and destroyed when it ends.

Global variables, on the other hand, live as long as the program runs.

```
x=10 # Global variable. lifetime is as long as the program runs.
def my_function():
    x=5 # local variable.lifetime is limited to the function only
    print("Inside Function:",x)
my_function()
print("Outside Function:",x)
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Inside Function: 5
Outside Function: 10
```

Using Global keyword.

```
x = 10  # Global variable

def modify():
    global x  # Use the global x (not a local one)
    x = 20    # Change the global x to 20
print(x)  # Prints 10 (original global value)
modify()  # Changes global x to 20
print(x)  # Prints 20 (global x was updated in modify())
OUTPUT
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
10
20
```

RESURSION/RECURSIVE FUNCTIONS:

A recursive function is a function that calls itself to solve a problem. It has two key parts.

1. Base Case: A condition to end the recursion and stop the function from calling itself indefinitely
2. Recursive Case: the part of the function where it calls itself to break the problem into smaller sub problems.

Example: factorial of a number.

factorial(5) # to slove fact(5) we need 5* fact(4)
= 5 * factorial(4) # to slove fact(4) we need 4* fact(3)
= 5 * 4 * factorial(3) # and so on
= 5 * 4 * 3 * factorial(2)
= 5 * 4 * 3 * 2 * factorial(1)
= 5 * 4 * 3 * 2 * 1 * factorial(0) # now we know fact(0) is 1. so we do backtracking to solve the factorial of 5
= 5 * 4 * 3 * 2 * 1 * 1 = 120

```
#Program to find the factorial of a number
def fact(n):
    if n == 0:              # Base case
        return 1
    else:
        return n * fact(n - 1)  # Recursive case

print(fact(5))  # Output: 120
```

```
#Program to find the Fibonacci series up to a limit
def fibo(n):
    if n == 0 or n == 1:
        return n
    else:
        return fibo(n - 1) + fibo(n - 2)

# Take input from user
num = int(input("Enter how many terms of Fibonacci series you want: "))
print("Fibonacci series:")
for i in range(num):
     print(fibo(i), end=" ")
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Enter how many terms of Fibonacci series you want: 6
Fibonacci series:
0 1 1 2 3 5
```

```
#Program to find gcd of two numbers
def gcd(a, b):
    if b == 0:
        return a
    else:
        return gcd(b, a % b)
# Call GCD function
print("GCD of", 48, "and", 18, "is:", gcd(48, 18))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
GCD of 48 and 18 is: 6
```

When to use Recursion and iteration:

Use iteration when:

You want better performance and efficiency.

The logic is straightforward and fits cleanly in a loop.

Use recursion when:

The problem can be broken into smaller subproblems.

Each step is a smaller version of the same problem.

It’s easier to express in recursive form (e.g., mathematical definitions).

You're working with trees, graphs, or backtracking.

MODULES IN PYTHON:

Python Module is a file that contains built-in functions, classes,its and variables.

There are many Python modules, each serves a different purpose.

Every file acts as a module in python

Modules let you break your code into smaller, reusable parts. You can use the import statement to bring these parts into your program and use the functions, variables, or classes they contain.

Types of Modules

- Built-in modules
- User-defined modules
- Third party modules

Built-in Modules:

These modules are part of the Python standard library and provide a wide range of functionalities, such as math, os, sys, datetime, and random.

Math:contains various mathematical functions for performing complex calculations

User-defined Modules:

These are custom modules created by users to organize their code and can be imported and used in other programs.

Third-party Modules:

These modules are external libraries that can be installed using package managers like pip and offer additional functionalities, such as NumPy, pandas, matplotlib, and requests.

How to use built in modules

We can import the functions, and classes defined in a module to another module using the import statement

When the interpreter encounters an import statement, it imports the module if the module is present in the search path.

Note: A search path is a list of directories that the interpreter searches for importing a module.

Syntax to Import Module in Python

import module_name

Ex: import math or import random

Note: This does not import the functions or classes directly instead imports the module only. To access the functions inside the module the dot(.) operator is used.

Syntax

module_name.Function_name

Example:

```
import math
print(math.sqrt(16))
print(math.factorial(5))
OUTPUT:
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
4.0
120
```

Know what a specific function does: Using help() function.

he help() function in Python is used to get information about anything, such as a function, module, class, or variable. It shows the documentation (docstring), which helps you understand what the specific thing does and how to use it.

Ex:

help("modules"): used to list all the available modules installed on your python environment.

Help("math"):used to list all the available functions in math modules

Example:sin(),cos(),

```
import math
help(math.gcd)
OUTPUT
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Help on built-in function gcd in module math:

gcd(x, y, /)
    greatest common divisor of x and y
```

Python Import From Module

Python's from statement lets you import specific attributes from a module without importing the module as a whole.

Syntax

From module_name import function_name/variable_name

Ex: here only gcd is imported. And it is imported without module prefix meaning we didn't use print(math.gcd(48,16)) instead we used print(gcd(48,16))

```
from math import gcd
print(gcd(48,18))#module prefix is not required.
```

If you try to use other functions without importing them,it will raise an error.

```
from math import gcd
print(gcd(48,18))
print(factorial(5))
```

```
OUTPUT
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
6
Traceback (most recent call last):
  File "f.py", line 3, in <module>
    print(factorial(5))
NameError: name 'factorial' is not defined
```

Now it wont show any error.since you are using gcd and factorial from math module.

```
from math import gcd,factorial
print(gcd(48,18))
print(factorial(5))
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
6
120
```

Import all Names

The * symbol used with the import statement is used to import all the names from a module to a current namespace.

If two modules have a function with the same name, they will overwrite each other. It's hard to know where a function or variable came from therefor generally its not recommended

Syntax:

from module_name import *

```
from math import *
print(pi)
print(sqrt(16)) #module prefix is not required.
OUTPUT
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
3.141592653589793
4.0
```

Renaming the Python Module

We can rename the module while importing it using the keyword.

Syntax: Import Module_name as Alias_name

```
import math as mt
print(mt.pi/2)
print(mt.exp(3/2))
#print(math.sqrt(16)) # this line will give error since mt is the module name now
print(mt.sin(20))
print(mt.log(10))
OUTPUT
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
1.5707963267948966
4.4816890703380645
0.9129452507276277
2.302585092994046
```

Random module:

It generates random numbers or picks random elements

random.randint():generates a random integer between the range

random.choice(): selects or picks a specific thing from a number of things

random.shuffle(): changes the position of the elements and shuffles them

random.uniform():generates a random floating point number between the range

```
import random
print(random.randint(1,20))
print(random.choice(['apple','banana','mango']))
colors=['red','pink','black','blue']
random.shuffle(colors)
print(colors)
print(random.uniform(1.0,2.5))
```

```
OUTPUT:
PS C:\Users> python -u "c:\Users\masrath
parveen\OneDrive\Desktop\tempCodeRunnerFile.py"
15
banana
['black', 'pink', 'red', 'blue']
1.0717588344338176
```

When to use which module syntax:

- If you are very sure about no conflicts go for Import*(all Names) module
- If you need specific item or if you know exactly what you will be needing then go for Import From Module
- Otherwise use general module : import module_name

User-defined Modules:

Create your own module.

- Every file acts as a module in python.

Step1:You create a module just by saving Python code in a file with a .py extension.

Step2:create another file. Now use the functions/variables etc from the first file into the current file using syntax : import module_name

Example:step1: creating own.py

```
# the name of this file is own.py
x=56
def add(a,b):
    return a+b
def sub(a,b):
    return a-b
def mul(a,b):
    return a*b
def fact(n):
    if n == 0:
        return 1
```

```
    else:
        return n * fact(n - 1) # Recursive case
```

Step 2:creating calculate.py

```
# the name of this file is calculate.py
import own # here own.py acts as a module
print(own.x) # from own.py we are printing the value of x
print(own.add(5,2)) # from own.py we are using add function
print(own.fact(5))
OUTPUT:
PS C:\Users\masrath parveen\PycharmProjects\div3> python calculate.py
56
7
120
```

You can use import from syntax as well.

```
from own import sub,mul
print(sub(9,2))
print(mul(2,5))
OUTPUT:
PS C:\Users\masrath parveen\PycharmProjects\div3> python calculate.py
7
10
```

```
import own
```

When you use import module_name, then python imports this module if its present in the python search path

Python search path:

It is the list of directories that a python interpreter search for the required module

You can see you search path using sys module

```
import sys
print(sys.path)
```

```
OUTPUT:
PS C:\Users\masrath parveen\PycharmProjects\div3> python calculate.py
['C:\\Users\\masrath parveen\\PycharmProjects\\div3', 'C:\\Users\\masrath
parveen\\AppData\\Local\\Programs\\Python\\Python38-32\\python38.zip',
'C:\\Users\\masrath parveen\\AppData\\Local\\Programs\\Python\\Python38-
32\\DLLs', 'C:\\Users\\masrath
parveen\\AppData\\Local\\Programs\\Python\\Python38-32\\lib', 'C:\\Users\\masrath
parveen\\AppData\\Local\\Programs\\Python\\Python38-32', 'C:\\Users\\masrath
parveen\\AppData\\Local\\Programs\\Python\\Python38-32\\lib\\site-packages']
```

Third-party Modules:

Packages:

A package is a collection of modules organized in a directory with a specific file named __init__.py (can be empty or contains initialization code) and it can have sub packages.

```
my_package/
│
├── __init__.py
├── module1.py
└── module2.py
```

Library:

A library is a collection of packages and modules.(used interchangeably with packages)

When to use which package

For AI and machine learning:

- ✓ Numpy
- ✓ Pandas
- ✓ Scipy

For visualization

- ✓ Matplotlib
- ✓ Seaborn

For deep learning:

- ✓ Scikit-learn
- ✓ Tenserflow
- ✓ Keras

For computer vision

- ✓ Open CV
- ✓ tenserflow

How to install packages:

You can install packages using PIP

PIP installs packages or preferred installer program:

It is a default package manager for python that allows you to install,manage,uninstall python packagesor libraries from the python package index(PyPI) which is a massive repositoryof python software.

PIP has access to PyPI which contains over 400,000 packages for various tasks like web development,data analysis,machine learning etc.

Check if PIP is installed

Open command prompt

Pip --version

Install packages using PIP:

Syntax: pip install package_name

Ex: pip install numpy

Uninstall packages using PIP:

Syntax: pip uninstall package_name

Ex: pip uninstall pandas

Upgrade packages using PIP:

Syntax: pip install --upgrade package_name

Ex: pip install --upgrade numpy

List all packages:

Syntax:pip list